

An Operator-Oriented Unary Calculus for Balanced Ternary and a Locality Obstruction for Addition

Status: prior-art-corrected publication draft.

Abstract

Balanced ternary and signed-digit arithmetic have long-standing theories of digit append, carry management, finite-state normalization, and arithmetic rewriting. In particular, Contejean, Marché, and Rabehasaina (CMR97) gave convergent rewrite systems for natural, integral, and rational arithmetic using balanced-ternary digit append. Against that background, we study the narrower operator language

$$I_a(x) = a + 3x, \quad D(n) = \frac{n - \text{lzd}(n)}{3}, \quad S(n) = 3n, \quad N(n) = -n.$$

Here I_a is the standard balanced-ternary digit-append map and D removes the least-significant digit. We formulate a one-hole unary term language generated by $\{D, I_a, S, N\}$ and formally verify in Lean that its finite rewrite system terminates and is confluent, that rewriting preserves integer-function semantics, and that distinct irreducible terms denote distinct maps $\mathbb{Z} \rightarrow \mathbb{Z}$.

We also isolate an exact state boundary at addition. Call a binary output H D -local if

$$H(x, y) = G(D(x), D(y))$$

for some $G : \mathbb{Z}^2 \rightarrow \mathbb{Z}$. We prove that the next-state output $H(x, y) = D(x + y)$ is not D -local. The minimal witness is $D(0) = D(1) = 0$, while $D(0 + 0) = 0$ and $D(1 + 1) = 1$. The exact identity

$$D(x + y) = D(x) + D(y) + \text{carry}(\text{lzd}(x), \text{lzd}(y))$$

identifies the discarded information. A named extension that pushes S through addition is not locally confluent at $D \circ S$. These results concern the stated open unary grammar and factorization notion. They do not claim priority for balanced-ternary arithmetic rewriting, finite-state carry, or parallel signed-digit addition, and they do not classify arbitrary rewrite systems for arithmetic.

1. Introduction

Every integer has a unique balanced-ternary expansion

$$n = \sum_{i \geq 0} a_i 3^i, \quad a_i \in \{-1, 0, +1\}.$$

This representation and its arithmetic are classical. Avižienis established the broad signed-digit principle that redundancy limits carry propagation and enables parallel arithmetic. Heuberger and Prodinger later analyzed carry propagation with automata in signed-digit systems, explicitly including balanced ternary. Frougny, Pelantová, and Svobodová developed general local and parallel addition algorithms and studied minimal supporting alphabets; subsequent work treated fixed blocks.

Arithmetic rewriting is also direct prior art. CMR97 represents an integer by balanced-ternary postfix digit append $x : a$, with $[x : a] = 3[x] + a$, and gives terminating and confluent arithmetic rewrite systems modulo associativity and commutativity of addition and multiplication. Therefore $I_a(x) = 3x + a$ in this paper is not a new section construction: it is the same integer map in operator notation. Likewise, D is the standard quotient that removes the least-significant balanced digit.

The remaining question is narrower:

Do the natural balanced-ternary unary operators form a canonical rewrite calculus, and what exact obstruction appears when addition is required to use the same local state?

The answer has four parts.

1. The unary language generated by $\{D, I_a, S, N\}$ has a canonical presentation on its specified one-hole syntax, including semantic injectivity of irreducible open terms.
2. The output $D(x + y)$ does not factor through $(D(x), D(y))$.
3. The balanced carry identity identifies the missing least-significant-trit information.
4. The natural carry-free push-in extension has a non-joining critical peak at $D \circ S$.

The first result is an operator-oriented formalization using standard rewriting theory. The second is the central state-factorization theorem. The third is known carry arithmetic in the present notation, and the fourth shows how that boundary appears in one named syntactic extension. The Lean development verifies the exact grammar, relation, and theorem statements; no claim is made that it is the first mechanization of signed-digit arithmetic.

No claim is made that addition lacks a finite presentation. Coefficient-word normalization is an explicit complete treatment of integer sums, but it retains normalization state absent from the unary tree calculus.

2. Balanced-ternary unary language

Let $\text{lsd}(n) \in \{-1, 0, +1\}$ be the balanced residue of n modulo 3. Define

$$D(n) = \frac{n - \text{lsd}(n)}{3}, \quad I_a(n) = a + 3n, \quad S(n) = 3n, \quad N(n) = -n.$$

Here $a \in \{-1, 0, +1\}$, and $I_0 = S$ as integer maps.

The formal term grammar is

```
t ::= var | D(t) | I-(t) | I0(t) | I+(t) | S(t) | N(t).
```

It is a one-hole unary grammar. It contains neither Add nor Mul, and it is not the operator-word string language used elsewhere in the project.

Evaluation assigns an integer function to each term:

$$[-]: \text{OpFrag} \rightarrow (\mathbb{Z} \rightarrow \mathbb{Z}).$$

For example,

$$[D(t)](n) = D([t](n)), \quad [I_a(t)](n) = I_a([t](n)).$$

Semantic equality below means equality as functions on every integer, not agreement at one input.

3. Canonical rewrite calculus

The rewrite relation is generated by the following root rules and closed under every unary context:

```
I0(t) → S(t) D(I-(t)) → t D(I0(t)) → t D(I+(t)) → t D(S(t)) → t N(N(t)) → t N(S(t)) → S(N(t))
N(I0(t)) → S(N(t)) N(I-(t)) → I+(N(t)) N(I+(t)) → I-(N(t)) N(D(t)) → D(N(t)).
```

Every rule is an exact identity of integer functions.

The final orientation is necessary for semantic canonicity. Without $N(D(t)) \rightarrow D(N(t))$, the terms $N(D(\text{var}))$ and $D(N(\text{var}))$ are distinct irreducibles with identical integer semantics.

3.1 Canonical-form theorem

Theorem 1 (canonical unary calculus). On OpFrag:

1. every rewrite sequence terminates;
2. the relation is locally confluent and hence confluent;
3. every term has a unique syntactic normal form;
4. rewriting preserves evaluation on $\mathbb{Z}$;
5. two irreducible terms with equal evaluation on every integer are syntactically equal.

Consequently, every term has a unique normal descendant, and that descendant is the unique irreducible representative of its integer function within this grammar.

The Lean façade theorem is `unary_complete_canonical_form`.

3.2 Proof structure

Termination uses the lexicographic rank

$$(\text{count}(l_0), N_{\text{inversions}}, \text{term size}).$$

An N -inversion is a pair consisting of an N -node and a pushable descendant among D, l_-, l_0, l_+, S . Cancellation decreases size, $l_0 \rightarrow S$ decreases the first coordinate, and moving N inward decreases the second.

The finite family of root and root-versus-inner critical pairs joins. Termination and local confluence give confluence by Newman's lemma. Semantic injectivity of irreducibles then follows from the normal-form grammar and uniqueness of balanced-ternary words.

The general proof method is standard. The project-specific object is the exact operator fragment, including the necessary orientation of N past D .

4. D -locality and the main theorem

4.1 Definition

A binary integer output $H : \mathbb{Z}^2 \rightarrow \mathbb{Z}$ is D -local if there exists $G : \mathbb{Z}^2 \rightarrow \mathbb{Z}$ such that

$$\forall x, y \in \mathbb{Z}, \quad H(x, y) = G(D(x), D(y)).$$

Thus H is D -local exactly when it factors through the pair of operand D -states. This is the only locality notion used here.

For addition, the relevant output is not $x + y$ itself but its next state

$$H_{\text{add}}(x, y) = D(x + y).$$

4.2 Exact non-locality

Theorem 2 (addition next-state non-locality).

$$\neg \exists G : \mathbb{Z}^2 \rightarrow \mathbb{Z} \quad \forall x, y, \quad D(x + y) = G(D(x), D(y)).$$

Proof. The inputs 0 and 1 have the same D -state:

$$D(0) = D(1) = 0.$$

Therefore the pairs (0, 0) and (1, 1) present the same argument (0, 0) to any proposed G . But

$$D(0 + 0) = 0, \quad D(1 + 1) = D(2) = 1.$$

No such G exists. QED

The Lean theorem is `add_not_DLocal`. Its predicate `DLocal` is the factorization definition above, applied to `fun x y => DZ (x + y)`.

The result is stronger than the failure of the single formula $D(x + y) = D(x) + D(y)$: it rules out every function of $(D(x), D(y))$ alone.

5. Addition and the missing carry

Balanced-ternary addition satisfies the exact identity

$$D(x + y) = D(x) + D(y) + \text{carry}(\text{lzd}(x), \text{lzd}(y)).$$

In Lean, the carry is the second component of `addDigit (lzd x) (lzd y)`. It belongs to $\{-1, 0, +1\}$.

The identity explains Theorem 2. The states $D(x)$ and $D(y)$ retain the higher parts of the operands after their least-significant trits have been removed. Those removed trits determine whether their sum leaves the digit set and produces a carry into the next position. Hence the carry is not an implementation accident: it is precisely the information absent from $(D(x), D(y))$.

At the minimal witness,

$$D(1 + 1) = D(2) = 1 \neq 0 = D(1) + D(1).$$

The Lean theorem `carry_of_add` is a rearrangement of the existing exact theorem `D_add`.

6. Natural carry-free extension and critical peak

Consider the syntax

```
t ::= x | y | D(t) | S(t) | Add(t, t)
```

with the named relation generated by

```
D(S(t)) → t S(Add(t, u)) → Add(S(t), S(u))
```

and congruence. There is deliberately no rule pushing D through `Add`, because the carry-free candidate is unsound.

The term

$$D(S(\text{Add}(X, Y)))$$

has two one-step descendants:

$$\text{Add}(X, Y)$$

by $D(S(t)) \rightarrow t$, and

$$D(\text{Add}(S(X), S(Y)))$$

by pushing S through `Add` inside D . Both descendants are irreducible in this named system, and they are syntactically distinct. Nevertheless, both evaluate to $x + y$.

Theorem 3 (named push-in obstruction). The named carry-free push-in relation is not locally confluent.

The Lean theorems are `pushIn_not_locally_confluent` and `pushIn_peak_semantic`.

This is a concrete obstruction to one natural extension. It is not a classification of every rewrite system containing addition, and it is not used to prove Theorem 2. Rather, it shows the same locality boundary appearing as a critical peak.

7. Restricted maximality question

Theorem 2 immediately excludes any exact first-digit implementation whose output $D(x + y)$ is required to be a function only of $(D(x), D(y))$. That is the precise locality boundary.

A stronger theorem about a class of complete tree rewrite systems would additionally require an independent syntactic definition of “carry-free” and a proof that completeness in that class forces D -locality. Defining the class by assuming D -locality, or by assuming that the two descendants in §6 remain irreducible, would only restate the conclusion. No such universal maximality statement is claimed here.

The broader project-level observation that the examined Add systems either remain incomplete or introduce constants, carry state, or coefficient normalization is retained as a human research record, not as a theorem of this note.

8. Related Work and Positioning

8.1 Balanced-ternary and signed-digit arithmetic

Balanced ternary is the radix-3 signed-digit system with alphabet $\{-1, 0, 1\}$. Unique finite representation of integers, digitwise negation, and the append operation $x \mapsto 3x + a$ are established background. Avižienis's 1961 work is the classical source for redundant signed-digit arithmetic: redundancy bounds carry propagation and supports parallel addition and subtraction. The broad arithmetic principle predates this paper by decades.

Our alphabet is nonredundant as an integer representation, but addition temporarily leaves it: $1 + 1 = 2$ requires a carry and a returned digit -1 . The identity in §5 expresses that familiar phenomenon through D and lsd . It is not presented as a new carry algorithm.

8.2 Rewrite systems for arithmetic

CMR97 is the central comparison. It gives algebraic presentations of naturals, integers, and rationals by convergent rewrite systems. Its integer representation uses balanced-ternary postfix digit append with digits $-1, 0, 1$:

$$[x : ; a] = 3[x] + a.$$

Its arithmetic system includes addition and multiplication and is terminating and confluent modulo their associativity and commutativity. Thus CMR97 already provides balanced-ternary arithmetic rewriting and predates this work. Walters and Zantema independently provide broader arbitrary-radix digit-application and integer-arithmetic TRSs; Bergstra and Ponse later place CMR97 within that digit-append tradition.

The present system solves a different, smaller rewrite problem: open one-hole unary terms over $\{D, l_a, S, N\}$, with equality interpreted as equality of functions $\mathbb{Z} \rightarrow \mathbb{Z}$. It contains no arithmetic constructor and uses no AC matching. Its strongest canonical form statement includes semantic injectivity of irreducible open terms. That difference is one of theorem scope and formal presentation, not a claim to have originated balanced-ternary arithmetic rewriting.

8.3 Finite-state carry and normalization

Heuberger and Prodinger study carry propagation in (q, d) signed-digit expansions, explicitly identify balanced ternary as the case $q=3, d=-1$, and construct automata recognizing the patterns that govern carry iterations. Finite-state descriptions of signed-digit carry are therefore established prior art.

The theorem `add_not_DLocal` does not assert that addition is not finite-state. It asserts only that the particular pair of quotient states $(D(x), D(y))$ loses information needed to determine the next quotient $D(x + y)$. The missing finite datum is exactly the pair of least-significant trits, or its induced carry. Adding such state is consistent with the classical automata literature.

8.4 Parallel and redundant numeration systems

Frougny, Pelantová, and Svobodová develop parallel addition for broad classes of non-standard numeration systems as a local digit-set conversion, generalizing the Avižienis approach. Their later work gives lower bounds and sharp constructions for minimal digit alphabets. Frougny, Heller, Pelantová, and Svobodová compare k -block with one-block

parallel addition and show that block processing can reduce the required alphabet.

Consequently, local normalization, finite-window arithmetic, parallelism, and alphabet minimality are mature topics. This paper neither proposes a parallel adder nor proves an alphabet-minimality result. Its locality definition is the specific factorization in §4, not the sliding-window locality used in parallel addition.

8.5 Relation to CMR97

The representation-level translation is exact:

$$x : {}_t\mathbf{a} \longleftrightarrow l_a(x), \quad [x : {}_t\mathbf{a}] = [l_a(x)] = 3x + a.$$

Repeated append corresponds to composition of sections. Conversely, every constructor-only composition of l_a 's can be written as a CMR97 digit-append term, with the written digit order chosen to preserve evaluation. The underlying integer representation is therefore equivalent.

D is the inverse quotient on the image of each section:

$$D(l_a(x)) = x.$$

The quotient interpretation is standard digit removal. What differs is that this paper makes D primitive and studies its interaction with append, shift, and negation as an open unary algebra. This compresses families of digit-level equalities into operator equations, but compression alone is not a novelty claim.

CMR97's arithmetic normalization theorems and this paper's canonical unary theorem are not interchangeable. CMR97 is broader in arithmetic operations; the present theorem is stronger only in its own orthogonal direction: for the specified open unary grammar, irreducible terms are injective under function semantics. The required orientation $N(D(t)) \rightarrow D(N(t))$, the exact `DLocal` non-factorization theorem, and the named push-in peak are results about this presentation. We did not find those exact statements in the bounded audit, but make no historical priority claim from that absence.

8.6 Comparison

Here “not found” means not found in the bounded audit of the cited source, not a universal absence claim. “Partial” marks a related object whose theorem scope differs.

Aspect	CMR97	Signed-digit/carry literature	This work
Integer representation	yes	yes	yes
Balanced ternary	yes	yes	yes
Primitive digit operations	yes	yes	yes
Addition	yes	yes	not central
Multiplication	yes	partial	not addressed
Quotient/ D operator	not found	partial	yes
Explicit <code>lsd</code> operator	not found	partial	yes
Section operators <code>I_a</code>	yes	yes	yes
Operator composition	not central	partial	yes
Term rewriting	yes	not central	yes
Confluence	yes	not central	yes
Termination	yes	not central	yes
Finite-state carry	not central	yes	partial

Residual dynamics	not addressed	partial	not addressed
Minimal behavioral quotient	not addressed	partial	not addressed
Lean formalization	not addressed	not addressed	yes
Automated invariant discovery	not addressed	not central	not addressed
Counterexample-first discovery	not addressed	not central	partial

The last three rows do not transfer the separate research-engine programme into this paper. “Counterexample-first” refers only to the explicit witnesses used to delimit the named rewrite claims.

No general rewriting theory, numeral system, signed-digit addition algorithm, carry automaton, or parallel arithmetic construction is claimed here.

9. Scope and limitations

The exact formal scope is:

- one-hole unary terms generated by $\{D, I_a, S, N\}$;
- integer-function semantics on $\mathbb{Z}$;
- output factorization through the pair $(D(x), D(y))$;
- the finite affine constructor class in Appendix A;
- the named push-in relation in §6.

The note does not claim:

- that addition has no finite rewrite presentation;
- that arbitrary finite Add-tree systems are incomplete;
- that every extension with additional state is a computer-algebra system;
- that the production operator-word table is confluent;
- that AC completion is impossible or unnecessary.

Coefficient-word normalization gives a complete representation of integer sums after evaluation. It is a different canonicalizer from the unary OpFrag tree system and is not imported into the locality proof.

10. Formal verification

The development uses Lean 4 and Mathlib. Its façade files are:

- `formal/BTCalculus/RewriteCore.lean`;
- `formal/BTCalculus/RewriteAddBoundary.lean`.

The principal theorem names are:

- `unary_complete_canonical_form`;
- `add_not_DLocal`;
- `carry_of_add`;
- `pushIn_not_locally_confluent`;
- `pushIn_peak_semantic`.

Supporting constructor results are `exactTriple_characterization`, `not_exact_Ip_Ip`, and `not_exact_Im_Im`. Lean also packages the boundary results as the secondary conjunction `add_requires_carry_state`.

The underlying unary proofs are in `OpFrag.lean`, `OpFragNewman.lean`, and `OpFragSemantic.lean`; the carry identity `D_add` is in `Algebra.lean`.

Verification command:

```
lake build BTCalculus
```

The formal files contain no `sorry`, `admit`, or added axioms. Lean verifies the stated concrete theorems; it is not itself the novelty claim.

11. Conclusion

The one-hole unary language generated by $\{D, I_a, S, N\}$ has a finite canonical rewrite presentation under exact integer-function semantics. Its normal forms are unique both syntactically and as representatives of the functions expressible in the grammar.

The next-state output of exact addition lies beyond the same local state model:

$$D(x + y) \neq G(D(x), D(y))$$

for every possible G . The carry identity identifies the missing least-significant-trit information, and the named push-in system shows the same boundary as a non-confluent critical peak. This is a precise locality obstruction, not an impossibility theorem for arithmetic.

Appendix A. Constructor-sum classification

Let

$$\mathcal{A} = \{S, I_+, I_-, N\}.$$

Write each constructor as $U(t) = s_U t + c_U$, where

$$(s_U, c_U) \in \{(3, 0), (3, 1), (3, -1), (-1, 0)\}.$$

Then

$$\forall x, y, \quad U(x) + V(y) = W(x + y)$$

holds exactly when

$$s_U = s_W, \quad s_V = s_W, \quad c_U + c_V = c_W.$$

The Lean theorem `exactTriple_characterization` gives the eight concrete triples $((S, S, S), (N, N, N), (I_+, S, I_+), (S, I_+, I_+), (I_-, S, I_-), (S, I_-, I_-), (I_+, I_-, S), (I_-, I_+, S))$. These are six parameterized rows when the two signs are written with I_a .

In particular,

$$I_+(x) + I_+(y) = S(x + y) + 2, \quad I_-(x) + I_-(y) = S(x + y) - 2.$$

The constants ± 2 are not balanced trits, so neither same-sign sum is represented by a constructor in $\mathcal{A}$.

Appendix B. Closed exploration record

The discovery phase also studied operator-word fragments. The full production word table is not locally confluent, while three named opt-in fragments have separate human Newman certificates. Those results are archived in

docs/theory/rewrite_calculus.md and are not part of the theorem package here. No word-table enlargement is proposed.

References and source map

1. A. Avižienis, “Signed-Digit Number Representations for Fast Parallel Arithmetic,” *IRE Transactions on Electronic Computers* EC-10(3) (1961), 389--400. doi:10.1109/TEC.1961.5219227.
2. É. Contejean, C. Marché, and L. Rabehasaina, “Rewrite Systems for Natural, Integral, and Rational Arithmetic,” in *Rewriting Techniques and Applications (RTA 1997)*, LNCS 1232, Springer (1997), 98--112. doi:10.1007/3-540-62950-5_64. See also the 1997 research report of the same title, Laboratoire de Recherche en Informatique, Université de Paris-Sud, Orsay.
3. C. Heuberger and H. Prodinger, “Carry Propagation in Signed Digit Representations,” *European Journal of Combinatorics* 24(3) (2003), 293--320. doi:10.1016/S0195-6698(03)00008-8.
4. C. Frougny, E. Pelantová, and M. Svobodová, “Parallel Addition in Non-Standard Numeration Systems,” *Theoretical Computer Science* 412(41) (2011), 5714--5727. doi:10.1016/j.tcs.2011.06.028.
5. C. Frougny, E. Pelantová, and M. Svobodová, “Minimal Digit Sets for Parallel Addition in Non-Standard Numeration Systems,” *Journal of Integer Sequences* 16 (2013), Article 13.2.17. primary text.
6. C. Frougny, P. Heller, E. Pelantová, and M. Svobodová, “ k -Block Parallel Addition versus 1-Block Parallel Addition in Non-Standard Numeration Systems,” *Theoretical Computer Science* 543 (2014), 52--67. doi:10.1016/j.tcs.2014.06.001.
7. H. R. Walters and H. Zantema, *Rewrite Systems for Integer Arithmetic*, Utrecht University report UU-CS-1994-43 (1994).
8. J. A. Bergstra and A. Ponse, “Three Datatype Defining Rewrite Systems for Datatypes of Integers Each Extending a Datatype of Naturals,” arXiv:1406.3280 (version 4, 2016).
9. M. H. A. Newman, “On Theories with a Combinatorial Definition of ‘Equivalence’,” *Annals of Mathematics* 43(2) (1942), 223--243. doi:10.2307/1968867.
10. F. Baader and T. Nipkow, *Term Rewriting and All That*, Cambridge University Press (1998).
11. D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, Addison-Wesley.

Canonical project records:

- docs/problems/rewrite_calculus.md;
- docs/theory/rewrite_calculus_prior_art.md;
- docs/theory/rewrite_calculus_formalization.md;
- docs/theory/rewrite_calculus_reviewer_packet.md;
- ledger rows BTC-op-fragment-nd-nf, BTC-op-fragment-nd-semantic, BTC-add-not-D-local, and BTC-push-in-S-peak.